

GRAPH DISTILLATION: TRANSFERRING ATTRIBUTION-GRAPH STRUCTURE FROM A LARGE TEACHER TO A SMALL STUDENT

Anonymous authors

Paper under double-blind review

ABSTRACT

Recent distillation objectives, whether reverse-KL, on-policy or decoupled, sharpen how a student imitates its teacher’s outputs, yet all are computed from those outputs alone: the student is graded on what it says, never on how it arrived there. Attribution graphs expose the how, tracing the direct linear effect of each internal component on the next in one forward pass. We introduce *graph distillation*, which builds attribution graphs over MLP neurons in teacher and student, coarsens each into a supergraph whose nodes carry functional labels shared by both models, and trains the student to route causal influence between those supernodes as the teacher does, via a Jensen–Shannon divergence on the supergraph’s edges. The graph loss simply adds to the standard KD loss. Supernodes are first built for two-digit addition by ANOVA over neuron activation grids, then generalised to a prompt-derived construction with no task-specific design. Distilling Llama-3-8B-Instruct into Llama-3.2-1B-Instruct, graph distillation lifts in-distribution accuracy from 65% to 98% and held-out accuracy on harder arithmetic from 22% to 53%, against 42% for supervised fine-tuning and 31–37% for output- and representation-level distillation at the same step count, and reaches its peak in half the steps standard distillation needs. Richer reasoning tasks are the natural next target.

1 INTRODUCTION

[Introduction. Why matching outputs need not transfer the teacher’s algorithm; why attribution graphs give a model-agnostic handle on the mechanism; contributions.]

2 RELATED WORK

[Related work. Knowledge distillation and feature/representation matching; mechanistic interpretability and circuit discovery (attribution patching, transcoders, attribution graphs); prior attempts at mechanism-level transfer.]

3 METHOD

3.1 OVERVIEW

Standard knowledge distillation constrains only the *output* of the student and says nothing about how that output was produced (Hinton et al., 2015). We add a second constraint on the *internal computation*: for a given arithmetic prompt we build an attribution graph over MLP neurons in each model, coarsen it into a small set of functionally-labelled *supernodes* that exist in both models, and require the teacher and the student to route causal influence between those supernodes in the same proportions.

Let $\mathcal{T}$ be the frozen teacher (Llama-3-8B-Instruct) and $\mathcal{S}$ the student (Llama-3.2-1B-Instruct) with trainable parameters $\theta_{\mathcal{S}}$. For a prompt x , the pipeline of Sections 3.2–3.4 produces one $K \times K$ supernode adjacency matrix per model, $W^{\mathcal{T}}(x)$ and $W^{\mathcal{S}}(x; \theta_{\mathcal{S}})$, whose row t says *where supernode*

t draws its input from. Supernodes are identified by a semantic label (“ones digit of the second operand”, “magnitude of the sum”, ...) rather than by neuron index, so the rows and columns of the two matrices correspond even though the models differ in width, depth, and neuron basis.

Dividing a row by its ℓ_1 norm turns it into a distribution over sources — supernode t ’s *routing profile*, the fraction of its incoming causal influence contributed by each other supernode. Writing P_t and Q_t for the teacher’s and the student’s routing profile for supernode t , and $\mathcal{A}$ for the supernodes present in both models, the graph loss is the mean Jensen–Shannon divergence between matched profiles,

$$\mathcal{L}_{\text{graph}}(\theta_S) = \frac{1}{|\mathcal{A}|} \sum_{t \in \mathcal{A}} \text{JSD}(P_t \parallel Q_t), \quad \text{JSD}(P \parallel Q) = \frac{1}{2} \text{KL}(P \parallel M) + \frac{1}{2} \text{KL}(Q \parallel M), \quad (1)$$

with $M = \frac{1}{2}(P + Q)$ and P_t detached, since the teacher is fixed. Unlike a one-sided KL, the JSD is symmetric — the student is penalised both for missing an edge the teacher uses and for inventing one the teacher does not — and bounded, so a single edge on which the two models place disjoint mass cannot dominate the loss.

The full objective adds (1) to the ordinary temperature-scaled KD term,

$$\mathcal{L}(\theta_S) = \underbrace{\frac{\tau^2}{|\mathcal{P}|} \sum_{p \in \mathcal{P}} \text{KL}\left(\text{softmax}(z_p^T/\tau) \parallel \text{softmax}(z_p^S/\tau)\right)}_{\mathcal{L}_{\text{KD}}} + \lambda \mathcal{L}_{\text{graph}}(\theta_S), \quad (2)$$

where $\mathcal{P}$ is the set of positions that predict an answer token or the end-of-sequence token after it, z_p the logits at position p , $\tau = 2$, and $\lambda = 0.1$ (Appendix C). The two gradients are close to orthogonal in parameter space throughout training (cosine within ± 0.09 and mean $+0.004$ over 500 logged steps), which in a space of 10^9 dimensions is simply what unrelated objectives look like; its one practical consequence is that under AdamW’s per-parameter normalisation the graph term takes full-size steps in the many directions where the KD gradient is near zero, so a term that is under 1% of the KD gradient’s norm can still change what the student learns.

3.2 ATTRIBUTION GRAPHS OVER MLP NEURONS

Our graph construction follows the circuit-tracing methodology of Ameisen et al. (2025); Lindsey et al. (2025): represent one forward pass as a directed weighted graph whose nodes are localised computational units — active at a specific layer and token position — and whose edge A_{ij} measures the *direct* linear effect of node j on node i , holding fixed everything off the direct path.

A node i is a triple (ℓ_i, p_i, n_i) : layer, token position, neuron index. It carries a *write vector* $v_i^{\text{out}} = a_i \cdot W_{\text{down}}[:, n_i]$, what the neuron adds to the residual stream, and a *read direction* $v_i^{\text{in}} = \partial a_i / \partial h$, the residual-stream direction it responds to. Edges are gradient–activation products, i.e. attribution patching (Nanda, 2023): for a target node i we form the scalar $f_i = \langle h_{p_i}^{(\ell_i)}, \text{sg}[v_i^{\text{in}}] \rangle$ — the linearised pre-activation of neuron i , with the read direction stop-gradiented so that only the *input* to the neuron is differentiated — and set

$$A_{ij} = \left\langle \nabla_{m_{p_j}^{(\ell_j)}} f_i, v_j^{\text{out}} \right\rangle, \quad (3)$$

where $m^{(\ell_j)}$ is the output of MLP block ℓ_j . Token-embedding nodes are handled the same way with v^{out} replaced by the embedding vector, and logit nodes use $f_c = \langle h_{-1}^{(L)}, W_U[:, c] - \overline{W_U} \rangle$ with the mean-subtracted unembedding column, following Ameisen et al. (2025). The logit targets c are the smallest set of top tokens whose probabilities under $\text{softmax}(z/\tau)$, $\tau = 2$, sum to 0.95, capped at ten. Since $n_{\text{pos}} \times d_{\text{mlp}}$ candidate neurons per layer is far too many, we keep the top 10% per layer by write-vector norm; activations at the BOS position are zeroed so the attention sink is never selected.

Why raw neurons are enough. The published pipeline attributes to cross-layer transcoder features rather than to neurons (Dunefsky et al., 2024). We attribute directly to raw MLP neurons: the construction (3) only requires a node’s residual-stream contribution to be linear in its activation, which neurons satisfy exactly, and it avoids the transcoder’s approximation error, which must otherwise be carried as uninterpretable *error nodes*. The remaining cost is polysemanticity, and that is precisely what the supernode step repairs — in the published work, too, the interpretable unit is a hand-built group of features rather than the individual feature. We automate that grouping (Section 3.3), and the loss (1) is defined over groups, never over individual nodes.

3.3 GROUPING NEURONS INTO SUPERNODES

Raw neuron graphs are far too large and too model-specific to compare across a 1B and an 8B model. Where supernodes in the published work are drawn by hand from feature visualisations, we replace the manual step with an automatic, purely functional criterion, which is what makes the grouping reproducible across two different models. We first describe the criterion concretely on our task (Section 3.3.1), then the task-agnostic construction it instantiates (Section 3.3.2).

3.3.1 CASE STUDY: ANOVA SUPERNODES FOR TWO-DIGIT ADDITION

Every neuron is evaluated on a fixed probe set: all two-digit addition prompts “ $a+b=$ ” for $a, b \in \{0, \dots, 99\}$, i.e. all 10^4 points of the 100×100 grid. This yields for neuron i an *activation grid* $T_i \in \mathbb{R}^{100 \times 100}$, with $T_i[a, b]$ its activation on that prompt — a direct empirical description of what the neuron responds to, as a picture. The node panels of Figure 1 are examples.

We then ask which of a fixed set of arithmetically meaningful partitions of the grid best explains that picture. Each candidate is a binary mask $M \in \{0, 1\}^{100 \times 100}$; for a prompt with operands (a^*, b^*) and sum $s^* = a^* + b^*$, the categories are

category	mask $M[a, b] = 1$ iff	category	mask $M[a, b] = 1$ iff
arg1 range	$a = a^*$	sum range	$a + b = s^*$
arg1 units	$a \equiv a^* \pmod{10}$	sum units	$a + b \equiv s^* \pmod{10}$
arg2 range	$b = b^*$	arg1 units and arg2 units	composite (see below)
arg2 units	$b \equiv b^* \pmod{10}$		

The score of mask M for neuron i is the fraction of the variance of T_i that the mask explains. For a binary regressor this is the squared Pearson correlation, and is exactly the one-way ANOVA η^2 between the two groups the mask induces:

$$s_{i,M} = \frac{(\langle T_i - \bar{T}_i, M - \bar{M} \rangle)^2}{\|T_i - \bar{T}_i\|_2^2 \cdot \|M - \bar{M}\|_2^2} \in [0, 1], \quad (4)$$

and a category’s score s_i^c is the best score over its masks. The composite category `arg1 units` and `arg2 units` instead scores $\min(s_i^{\text{arg1 units}}, s_i^{\text{arg2 units}})$, so it fires only for a neuron sensitive to both ones digits at once. Neurons are ranked not by s_i^c but by *specificity*, the margin over the best competing category, $\delta_i^c = s_i^c - \max_{c' \neq c} s_i^{c'}$, and supernode S_c is the top $n_{\text{per}} = 10$ neurons by δ_i^c . Ranking by margin is what keeps the supernodes functionally disjoint: a neuron firing on a broad band of a scores well on several correlated categories at once, and only the margin separates a genuine ones-digit detector from a magnitude detector that happens to correlate with one.

Two categories are handled differently, because their neurons are defined by what they *write* rather than by what they *read*. For `sum range` and `sum units` we rank candidates by how closely the neuron’s direct logit attribution $\text{softmax}(v_i^{\text{out}^\top} W_U)$, restricted to the number tokens 0–200, matches the target distribution in KL — a point mass at s^* for `sum range`, and uniform over $\{n : n \equiv s^* \pmod{10}\}$ for `sum units`. The same score defines two further `dla` supernodes, one per sum target, collecting the neurons that write those distributions to the output most directly; these are the bottom two nodes on each side of Figure 1.

Because the same category list produces the same supernode labels in both models, the teacher and student supergraphs are aligned automatically; training requires the two label sets to match exactly and falls back to their intersection with a warning if they do not.

3.3.2 THE GENERAL RECIPE: SUPERNODES FROM LOGIT AND ARGUMENT ATTRIBUTION

The ANOVA categories above are hand-written for two-digit addition, but nothing downstream of the grouping depends on where the labels come from: the loss (1) only needs supernode labels that are computable in both models and identical across them. Our general construction derives the labels from the prompt itself — one supernode per input token, plus one for the output distribution — so it transfers to a new task with no masks or probe grids to design.

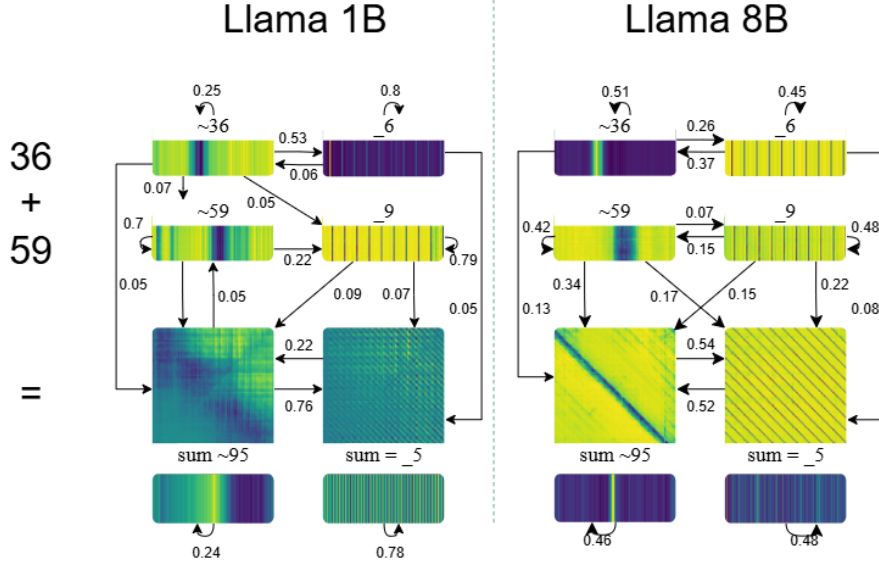


Figure 1: Supergraphs for the prompt “36+59=” in Llama-3.2-1B (left) and Llama-3.1-8B (right). Each node is a supernode from Section 3.3, drawn as the activation grid T_i of one representative member neuron — its activation across all 10^4 two-digit addition prompts, with operand a on one axis and b on the other — so that membership and computational role can be read off the same picture. Nodes ~ 36 and ~ 59 are `arg1/arg2 range`, `_6` and `_9` are `arg1/arg2 units`, `sum ~95` and `sum =_5` are `sum range` and `sum units`, and the bottom pair are the corresponding `dla` supernodes. Edge labels are the supernode weights of (5); self-loops are within-supernode recurrence. The 8B grids are visibly cleaner: its `sum ~95` node is a sharp anti-diagonal ($a + b \approx 95$) and its `sum =_5` node a clean periodic lattice ($a + b \equiv 5 \pmod{10}$), whereas the 1B counterparts are diffuse and noisy — the same structure, represented less separably in the smaller model.

Argument supernodes. For every token position q (BOS excluded) with embedding E_q , neuron i is scored by $|\langle v_i^{\text{in}}, E_q \rangle|$ — an inner-product proxy for the direct attribution edge (3) from the token-embedding node at q into the neuron’s linearised pre-activation, costing no extra backward pass. Each neuron’s scores are then normalised across positions into a distribution, so a neuron counts toward a position only through the *share* of its total embedding alignment concentrated there; this favours neurons selectively tied to one token over neurons whose read direction is simply large. The supernode for position q is the top n_{per} neurons by that share, and its label is the token string itself (`arg:36`, `arg:+`, `arg:59`, `arg:=`).

The output supernode. At the answer position, each neuron’s direct logit attribution $\text{softmax}(v_i^{\text{out}^\top W_U)$ is scored by its KL divergence from the reference output distribution $\text{softmax}(z/\tau)$, restricted to the reference’s top 100 tokens and renormalised. The n_{per} closest neurons form a single `dla` supernode — the neurons that most directly write the answer distribution — generalising the DLA ranking used for the sum categories above from a hand-specified target to the model’s actual output. During training the reference logits z are the teacher’s for both models (Appendix A), which anchors the student’s `dla` supernode to a stationary target.

Every label is derived from the prompt tokens and reference logits the two models share, so the teacher and student supergraphs align automatically on any prompt. This construction is the default in our training runs; the ANOVA categories of Section 3.3.1 remain available as an opt-in whitelist, and produce the task-level supergraphs of Figure 1.

3.4 SUPERNODE EDGE WEIGHTS

The supergraph’s edges are the supernode adjacency of Ameisen et al. (2025),

$$\text{supernode_adjacency}(t, s) = \frac{\sum_{n_t \in N_t} \text{frac_external}(n_t) \cdot \sum_{n_s \in N_s} A'_{n_t, n_s}}{\sum_{n_t \in N_t} \text{frac_external}(n_t)}, \quad (5)$$

where N_t and N_s are the member nodes of supernodes t and s , and A' is the graph adjacency with each node’s incoming weights normalised to sum to one over *all* sources — neurons, token embeddings and logits alike. The weight $\text{frac_external}(n_t)$ is the fraction of a member’s absolute inbound weight that comes from outside its own supernode, so members that mainly feed one another count for little and a supernode cannot dominate its own inbound profile. Because A' normalises over all nodes while the supergraph spans only supernode columns, the rows of `supernode_adjacency` do not sum to one, which is why the routing profiles of Section 3.1 renormalise them.

Implementation details — adapting the attribution pipeline to Llama’s gated MLPs and Hugging Face autograd, and the optimisations that make it fast enough for a training loop — are given in Appendices A and D.

4 EXPERIMENTAL SETUP

The teacher is Meta-Llama-3-8B-Instruct and the student Llama-3.2-1B-Instruct, both public checkpoints; the teacher is frozen and runs in bf16, the student holds fp32 master weights under a bf16 autocast forward (Appendix A). Training prompts are two-digit additions “ $a+b=$ ” (22_add), batch 32, AdamW at 10^{-6} with no schedule, 100 steps. Generalisation is measured on four families never seen in training: three- and four-operand sums of two-digit numbers (222_add, 2222_add), three-digit addition (33_add) and two-digit by one-digit multiplication (21_mult), 5000 held-out prompts each (1000 for 21_mult), scored by greedy decoding and exact match on the leading integer of the continuation. We compare three objectives through one trainer stack: supervised fine-tuning on the gold answers (SFT); standard KD from the teacher’s logits at $\tau = 2$; KD plus the graph loss (2) with $\lambda = 0.1$ and the prompt-derived supernodes of Section 3.3.2; and, as representation-level references that constrain the student’s internals without a graph, KD plus the TinyBERT recipe (Jiao et al., 2020) — mean-squared error between a learned linear projection of the student’s residual stream and the teacher’s at uniformly mapped layers, plus attention-pattern MSE at the same layers — and KD plus linear centred kernel alignment (Kornblith et al., 2019) on the residual streams, both at weight 0.1 in place of λ . Every baseline carries three seeds and the distillation baselines train for 300 steps so that their later peaks are not cut off. The full configuration is Table 6.

[Still to add: hardware and wall-clock cost per step for each objective.]

5 RESULTS

Table 1 and Figure 2 compare the objectives. Graph distillation reaches 0.983 in-distribution and 0.525 mean held-out accuracy at step 100, against 0.976 ± 0.028 and 0.415 ± 0.025 for SFT and 0.900 ± 0.011 and 0.369 ± 0.022 for standard KD. The held-out margin over SFT is 0.11, about four SFT seed standard deviations, and holds on three of the four families; on 33_add SFT is ahead at step 100, with the largest seed spread of any cell in the table. The margin over standard KD is 0.16 at matched steps and 0.14 against the control’s later peak at step 160, and graph distillation reaches its own peak at step 85. On 21_mult, the one held-out family that is not addition, graph distillation nearly matches the teacher’s in-distribution accuracy while every baseline stops near 0.55.

Constraining the student’s internals is not by itself what helps. The two representation-level baselines, which add a hidden-state loss to the same KD term at the same weight, land at or below standard KD: 0.309 ± 0.003 mean held-out accuracy at step 100 for the TinyBERT recipe and 0.341 ± 0.004 for CKA, with peaks of 0.375 and 0.373 against the control’s 0.383, and in-distribution accuracy within 0.02 of the control throughout. Both dip exactly as standard KD does. What the graph term adds is therefore not a generic regulariser on the residual stream but something specific to the routing target, and Appendix C tests that directly by scrambling the target.

Table 1: Accuracy at step 100 on the training family (22_add) and the four held-out families, the mean over the held-out families, and the peaks over the run with their step. Seed mean $\pm$ sd over three seeds for the baselines; the graph run is a single seed. The three distillation baselines trained to step 300; the teacher scores 0.957 on 22_add. Bold is the best mean in a row; italics are below the untrained student. *Dip / recovery* is the minimum in-distribution accuracy over steps 1–40 and the first later step back at or above the untrained student.

	untrained student	standard KD (3 seeds)	TinyBERT (3 seeds)	CKA (3 seeds)	SFT (3 seeds)	graph distillation ($\lambda = 0.1$, 1 seed)
22_add (train)	0.653	0.900 ± 0.011	0.890 ± 0.012	0.889 ± 0.010	0.976 ± 0.028	0.983
222_add	0.190	0.370 ± 0.019	0.359 ± 0.009	0.358 ± 0.017	0.397 ± 0.049	0.470
2222_add	0.046	0.085 ± 0.009	0.085 ± 0.004	0.083 ± 0.009	0.092 ± 0.015	0.113
33_add	0.492	<i>0.491 $\pm$ 0.010</i>	<i>0.487 $\pm$ 0.004</i>	<i>0.477 $\pm$ 0.009</i>	0.614 $\pm$ 0.086	0.579
21_mult	0.135	0.530 ± 0.049	0.512 ± 0.022	0.517 ± 0.038	0.558 ± 0.048	0.938
mean OOD	0.216	0.369 ± 0.022	0.361 ± 0.009	0.359 ± 0.018	0.415 ± 0.025	0.525
mean OOD, peak (step)	—	0.383 ± 0.007 (160)	0.375 ± 0.011 (125)	0.373 ± 0.004 (160)	0.440 ± 0.002 (60)	0.532 (85)
22_add, peak (step)	—	0.947 ± 0.002 (210)	0.931 ± 0.007 (195)	0.938 ± 0.002 (280)	0.993 $\pm$ 0.001 (65)	0.986 (70)
dip / recovery	—	0.248 / 60	0.238 / 55	0.246 / 60	0.505 / 20	0.493 / 25

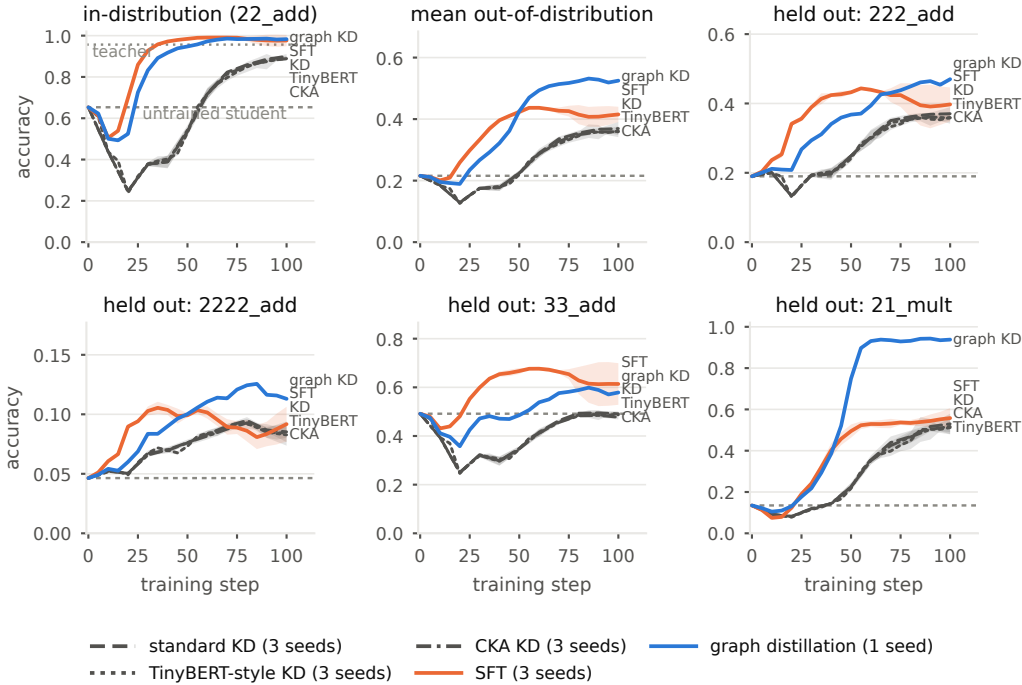


Figure 2: Accuracy against training step: in-distribution, the mean over the four held-out families, and each family. The three distillation baselines share the grey of the reference method and differ in line style; seed means with a ± 1 sd band for the three-seed baselines. The dashed rule is the untrained student and the dotted rule the teacher. Every run dips below the untrained student before recovering; the dip is deepest and longest for standard KD and the graph term removes most of it (Appendix C).

Standard KD is the weaker baseline on every measure. It falls to 0.25 in-distribution in its first twenty steps and needs sixty to recover the untrained student, where SFT and graph distillation dip to about 0.50 and recover by step 25; it never reaches SFT’s held-out peak in three times the steps. The graph term is what closes that gap: Appendix C shows the dip shrinking monotonically with λ and the held-out gain appearing only once a run has recovered. Appendix B shows that the choice of linearisation inside the attribution does not change the outcome at the working weight.

[pending: seeds 2–3 for graph distillation, on which the error bars of its row depend.]

[Still to add: graph-loss and KD-loss curves; whether supernode routing converges toward the teacher's; per-supernode breakdown of which routes align and which do not.]

6 ANALYSIS

[Analysis. Qualitative before/after supergraphs for the student; do the student's activation grids sharpen over training? Ablations beyond λ (Appendix C) and the linearisation (Appendix B): JSD vs. other divergences, number of supernodes, n_{per} .]

7 LIMITATIONS

[Limitations. Single task family (two-digit addition) and a single teacher/student pair; attribution graphs are built per prompt and only at sampled positions; edge sign is discarded by the input normalisation; supernode membership is recomputed but not itself differentiable.]

8 CONCLUSION

[Conclusion.]

AI USE STATEMENT

[AI use statement (required by ICLR 2027; see the AI Policy for Authors). State which of the tasks with required disclosure generative AI was used for and which it was not, how AI-assisted work was reviewed, and that the authors take responsibility for the final content.]

ETHICS STATEMENT

[Ethics statement (recommended). Synthetic arithmetic data, public model checkpoints, no human subjects; note anything the Code of Ethics asks about.]

REPRODUCIBILITY STATEMENT

[Reproducibility statement (recommended). Point to the appendices that carry the full hyperparameters, the training precision regime, the supernode construction and the diagnostics, and to the anonymised code in the supplementary material.]

REFERENCES

- Emmanuel Ameisen, Jack Lindsey, Adam Pearce, Wes Gurnee, Nicholas L. Turner, Brian Chen, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Thomas Conerly, Joshua Batson, Chris Olah, and Tom Henighan. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*, 2025. URL <https://transformer-circuits.pub/2025/attribution-graphs/methods.html>.
- Jacob Dunefsky, Philippe Chlenski, and Neel Nanda. Transcoders find interpretable LLM feature circuits. *Advances in Neural Information Processing Systems*, 2024.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network, 2015.
- Xiaoqi Jiao, Yichun Yin, Lifeng Shang, Xin Jiang, Xiao Chen, Linlin Li, Fang Wang, and Qun Liu. TinyBERT: Distilling BERT for natural language understanding. In *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 4163–4174, 2020.
- Simon Kornblith, Mohammad Norouzi, Honglak Lee, and Geoffrey Hinton. Similarity of neural network representations revisited. In *Proceedings of the 36th International Conference on Machine Learning*, pp. 3519–3529, 2019.

Jack Lindsey, Wes Gurnee, Emmanuel Ameisen, Brian Chen, Adam Pearce, Nicholas L. Turner, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Thomas Conerly, Joshua Batson, Chris Olah, and Tom Henighan. On the biology of a large language model. *Transformer Circuits Thread*, 2025. URL <https://transformer-circuits.pub/2025/attribution-graphs/biology.html>.

Neel Nanda. Attribution patching: Activation patching at industrial scale. <https://www.neelnanda.io/mechanistic-interpretability/attribution-patching>, 2023.

Noam Shazeer. GLU variants improve transformer. In *arXiv preprint arXiv:2002.05202*, 2020.

A ADAPTING THE PIPELINE TO LLAMA

The published methodology assumes a transcoder replacement model and a TransformerLens-style hook API (Ameisen et al., 2025). Running it inside a Hugging Face training loop on Llama required several changes, most forced by the requirement that the student’s graph carry gradients back into θ_S .

Gated MLPs. Llama’s MLP is a SwiGLU block (Shazeer, 2020) with no single input matrix. With residual-stream input h at the node’s position,

$$a_i = \sigma(g_i) u_i, \quad v_i^{\text{in}} = \frac{\partial a_i}{\partial h} = \sigma(g_i) W_{\text{up}}[n_i, :] + u_i \sigma'(g_i) W_{\text{gate}}[n_i, :], \quad (6)$$

where $g_i = \langle W_{\text{gate}}[n_i, :], h \rangle$, $u_i = \langle W_{\text{up}}[n_i, :], h \rangle$ and σ is SiLU. The read direction is therefore a product-rule sum of two terms that itself depends on the activation, rather than a fixed row of W_{in} .

Differentiability. The pre-selection top- k is detached (indices are not differentiable), but activations and read/write vectors stay in the ambient autograd context, so gradients flow $\mathcal{L}_{\text{graph}} \rightarrow \text{supernode_adjacency} \rightarrow A' \rightarrow A \rightarrow v^{\text{out}} \rightarrow W_{\text{down}}$ back into the model. Supernode *membership* is recomputed each step but treated as fixed structure; only the aggregation (5) is rebuilt differentially, out-of-place, so that autograd views survive.

Shared attribution targets. Both models are attributed to the *same* logit token set — the teacher’s — and the student’s d1a supernodes are selected against the teacher’s logits. Otherwise the two graphs would be built over different output nodes, and would be non-stationary early in training when the student’s own output distribution is still wrong.

Linearisation. The published method freezes attention patterns and normalisation denominators when linearising, so that edges reflect only the direct residual-stream path. We implement both freezes as backward-only stop-gradients and make each independently selectable, so (3) can be evaluated either with the direct-path linearisation or through the full Hugging Face Jacobian. Appendix B measures what the choice costs at the working λ : at initialisation it rotates the graph gradient by 40° to 50°, and over a full run it changes downstream accuracy by less than the seed-to-seed spread of the baselines.

Precision. The graph gradient is small next to the KD gradient — under 1% of its norm at step one and a few per cent by the end of a run for $\lambda \in [0.03, 0.3]$ (Table 4) — and it arrives one prompt at a time, since each prompt’s graph is backpropagated and freed before the next is built (Appendix D). With bf16 parameters those ~ 32 partial sums each land below half a unit in the last place of a `.grad` buffer that already holds the KD gradient, and are rounded away: we measured only 66–100% of the graph gradient surviving accumulation, decreasing with λ , which silently compressed a λ sweep toward its top. bf16 weights lose the update itself as well, since an AdamW step at 10^{-6} is under half a ULP for most weights; only about 2% of the parameters changed on a step. The student therefore holds fp32 master weights and fp32 optimiser moments and runs its forward under bf16 autocast; the inference-only teacher stays bf16. Every run records the fraction of parameters that moved on step one as a canary (it is 1.000 for every run reported here), and the step-one check of Appendix C confirms that scale-invariant summaries of the graph gradient agree across λ .

B THE DIRECT-PATH LINEARISATION AS A TRAINING SIGNAL

Circuit tracing freezes attention patterns and RMSNorm denominators so that an edge is the contribution of the direct path alone (Ameisen et al., 2025). That choice is made for interpretability: it renders the graph locally linear in the node activations and therefore readable. It is not obvious that it is also the right choice when the graph is a regression target rather than a figure, since the frozen edge omits real causal influence — paths on which a source node shifts *where* a target attends, and the rank-one RMSNorm term that projects out the component of each edge along the activation direction. This appendix measures the difference, first in the gradient the choice produces and then in the student it trains.

B.1 IMPLEMENTATION

Both freezes are backward-only. Detaching a multiplicative factor leaves every forward value bit-identical, so activations, read and write vectors, neuron pre-selection and supernode membership are unchanged; only the gradients that become edge weights differ. Freezing attention additionally requires a kernel that materialises the pattern, since the default fused kernel never forms it.

One implementation detail is load-bearing. Attribution involves two forward passes: one whose captured MLP inputs carry gradient back to θ_S , and one whose *backward* produces the edge values of (3). The freeze belongs only on the second. Applied to the first it cannot change any edge — being backward-only, it changes no forward value — but it silently biases training, zeroing the gradient to W_Q and W_K outright and stripping the rank-one correction from every RMSNorm on the path back to the weights. We hit this: with the freeze on both passes, in-distribution accuracy collapsed from 0.634 to 0.090 within twenty steps while the KD loss continued to fall.

The RMSNorm freeze has a second, measurable effect on the *target*. Frozen minus true Jacobian is $+r \hat{x} \hat{x}^\top$, a rank-one radial term, and in Llama $\hat{x}$ is dominated by the massive activations at the attention sink. Frozen edges therefore load onto the BOS node (its share of attribution mass rises from 0.11 to 0.47 in the teacher and from 0.07 to 0.53 in the student), which saturates every member’s `frac_external` near 1 and collapses the member weighting of (5) to a plain mean. The ablation below therefore includes an arm that keeps the unfrozen Jacobian but sets the weighting to a constant, to separate the freeze’s effect on the target from its effect on the gradient.

B.2 GRADIENT GEOMETRY AT INITIALISATION

Table 2 reports the graph gradient under each variant at the student’s initial checkpoint, against the unfrozen gradient and against the KD gradient.

Table 2: Graph-loss gradient under each linearisation, at the student’s initial checkpoint; 22_add, 8 graph prompts, $\lambda_{\text{graph}} = 1$. Cosines are against the unfrozen graph gradient and against the KD gradient. Measured with the earlier bf16 gradient accumulation (Appendix A), which attenuates the graph gradient’s magnitude but not its direction. *[pending: re-measure in the fp32 regime.]*

	$\mathcal{L}$	$\ g\ $	$\ g\ /\ g_{\text{unfr}}\ $	$\cos(g, g_{\text{unfr}})$	$\cos(g, g_{\text{KD}})$
$\mathcal{L}_{\text{KD}}$	2.0254	50.41	—	—	—
unfrozen	0.3405	3.647	1.000	+1.0000	−0.0236
attention frozen	0.3501	4.059	1.113	+0.6276	−0.0103
RMSNorm frozen	0.3722	3.991	1.094	+0.7652	−0.0153
both frozen	0.3745	3.856	1.057	+0.6984	−0.0082

The freeze changes direction, not strength: every frozen variant is 40° to 51° away from the unfrozen gradient while being slightly *larger* in norm. The unfrozen total-effect gradient is the smallest of the four, which is what destructive interference between paths produces — freezing removes the cancellation rather than removing signal. The two freezes also partially cancel each other: freezing both gives a smaller gradient than either alone and an angle between the two. A plausible mechanism is that the RMSNorm freeze injects the radial sensitivity described above, while the attention freeze deletes the QK path, the channel most responsive to a radial perturbation because rescaling the query

sharpens or flattens the softmax. Whether a 45° rotation of a gradient this small matters to training is the question the next section answers.

B.3 TRAINING ABLATION

Table 3 and Figure 3 train the variants for 100 steps at the working weight $\lambda = 0.1$ under otherwise identical settings (those of Appendix C), evaluating in-distribution on two-digit addition and out-of-distribution on four arithmetic families never seen in training.

Table 3: Distillation accuracy under each linearisation at $\lambda = 0.1$, step 100, laid out as Table 1. Single seed per arm; the two arms not yet run are marked.

	untrained student	unfrozen	attention frozen	RMSNorm frozen	both frozen	unfrozen, constant wt.
22_add (train)	0.653	0.983	[pending]	[pending]	0.984	0.986
222_add	0.190	0.470	[pending]	[pending]	0.430	0.479
2222_add	0.046	0.113	[pending]	[pending]	0.095	0.117
33_add	0.492	0.579	[pending]	[pending]	0.579	0.591
21_mult	0.135	0.938	[pending]	[pending]	0.938	0.944
mean OOD	0.216	0.525	[pending]	[pending]	0.511	0.533
mean OOD, peak (step)	—	0.532 (85)	[pending]	[pending]	0.530 (85)	0.538 (85)
22_add, peak (step)	—	0.986 (70)	[pending]	[pending]	0.987 (70)	0.987 (70)
dip / recovery	—	0.493 / 25	[pending]	[pending]	0.509 / 25	0.511 / 25

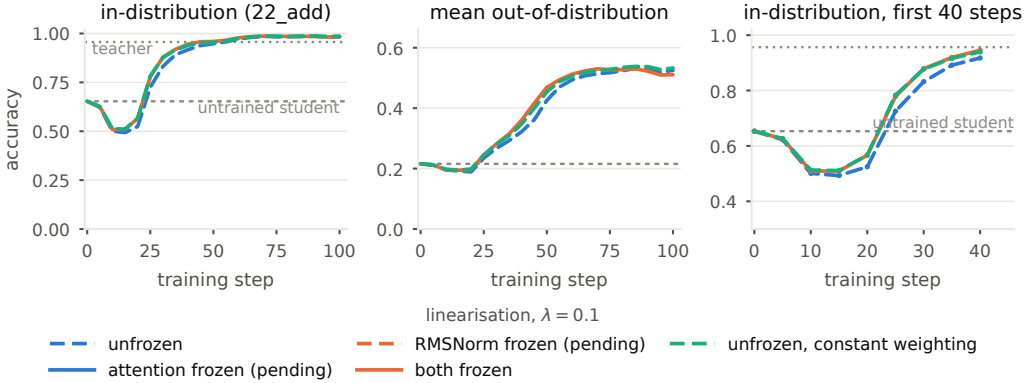


Figure 3: Accuracy against training step for the linearisation arms at $\lambda = 0.1$. Hue carries the RMSNorm freeze and line style the attention freeze, so the 2×2 design reads as two pairs; the constant-weighting arm is the third hue. *Left, centre*: the full run; *right*: the first forty steps of the in-distribution curve, where the arms differ most. The dashed rule is the untrained student and the dotted rule the teacher. The arms lie on top of one another: that overlap is the result.

At the working weight the linearisation does not change what the student learns. The three arms run so far end within 0.022 mean OOD of one another at step 100 and within 0.008 at their peaks, which all fall at step 85; their in-distribution accuracies are within 0.003. That spread is smaller than the seed-to-seed standard deviation of the baselines in Section 5 (0.025 mean OOD for SFT over three seeds), so no ordering among them can be claimed from one seed. Two things do separate the arms. Both the freeze and the constant weighting shallow the dip that every run takes before recovering (Appendix C): the minimum rises from 0.493 to 0.509 and 0.511, and the both-frozen arm reaches it five steps sooner. And the both-frozen arm is the only one to lose ground after its peak, giving back about 0.03 on each of 222_add and 2222_add between steps 70 and 100 where the other two are flat or still rising.

The constant-weighting arm is the informative null. It reproduces the unfrozen arm to within 0.008 on every family, so the `frac_external` weighting of (5) is inert here, and by the same token the

RMSNorm freeze’s saturation of that weighting cannot be what any RMSNorm-frozen arm is doing. Whatever the freeze contributes must come through the gradient, and at this λ that contribution is inside noise.

This reverses an earlier reading. An ablation at $\lambda = 1$, 15 steps, under the bf16 accumulation of Appendix A, had ranked the arms monotonically with the amount frozen. That measurement ordered the arms by the depth of the dip — at 15 steps there is no OOD signal yet, since no run has recovered the untrained student — and, as Table 3 shows, the freeze does shallow the dip. It is the extrapolation from the dip to generalisation that did not hold.

[pending: the attention-only and RMSNorm-only arms at $\lambda = 0.1$; seeds 2–3 for the unfrozen and both-frozen arms; one both-frozen run at $\lambda = 0.3$, where the graph gradient is three times larger and a rotation of it would have more room to matter.]

B.4 GRADIENT DYNAMICS OVER TRAINING

Every run records per-step gradient instrumentation. Figure 4 plots it and Table 4 summarises it.

Table 4: Per-step gradient metrics averaged over the 100 steps, $\lambda = 0.1$. The graph norm is already scaled by λ , so these are the quantities that reach the optimiser; the cosine column gives the signed mean and, in parentheses, the mean absolute value.

	$\ g_{KD}\ $	$\lambda\ g_{\text{graph}}\ $	ratio	cos (mean cos)	sign flips
unfrozen	10.46	0.208	0.042	+0.0040 (0.019)	0.0160
attention frozen	<i>[pending]</i>	<i>[pending]</i>	<i>[pending]</i>	<i>[pending]</i>	<i>[pending]</i>
RMSNorm frozen	<i>[pending]</i>	<i>[pending]</i>	<i>[pending]</i>	<i>[pending]</i>	<i>[pending]</i>
both frozen	10.48	0.211	0.041	+0.0030 (0.020)	0.0167
unfrozen, constant weighting	10.46	0.213	0.040	+0.0035 (0.019)	0.0158
spread across arms	0.3%	2.1%	5.4%	—	5.6%

The graph term progressively takes over. $\|\nabla\mathcal{L}_{KD}\|$ falls from 49.2 at step one to 2.5 at step 100, while $\lambda\|\nabla\mathcal{L}_{\text{graph}}\|$ goes from 0.30 to 0.17. Their ratio therefore climbs from 0.006 to 0.07, with transient spikes to 0.27 on individual batches, and the fraction of parameter entries whose update sign the graph term decides rises from 0.3% to 2.3%. Nobody chose this schedule: it is a consequence of KD converging faster than the graph objective. It also lines up with the accuracy curves of Appendix C: in-distribution accuracy is saturated by step 70 in every graph run while mean OOD accuracy goes on rising to step 85.

No aggregate gradient statistic separates the arms, which is consistent with their identical accuracy: the run means differ by 0.3% in KD norm, 2% in graph norm and about 5% in ratio and sign flips, and the cosine traces coincide step for step. If the arms differ at all it is in *which* parameters the graph term moves, and scalar summaries are blind to that by construction.

One incidental observation: the pre-clip gradient norm never falls below 1.9 and so exceeds the clip threshold of 1.0 at every step of every run. Gradient clipping is not a rarely-triggered safety valve here but a permanent rescaling — one that AdamW largely undoes, since per-parameter normalisation cancels a uniform scale factor.

B.5 CAVEATS

Single seed per arm; one supernode construction, the arg-token and DLA supernodes of Section 3.3.2, not the ANOVA supernodes. The freeze acts on the edge coefficients, before supernodes are formed, so the RMSNorm result should transfer; the attention result is less safe to extrapolate, since arg-token supernodes are positional and attention is the only mechanism that moves information between positions. The gradient geometry of Table 2 was measured in the earlier bf16 regime. 2222_add is close to unlearned in every configuration.

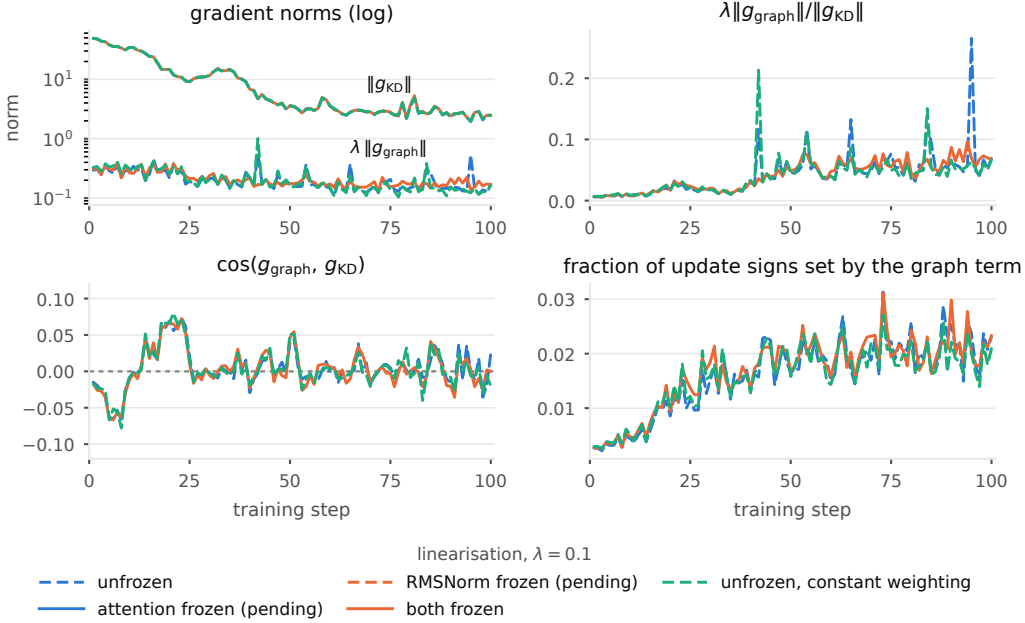


Figure 4: Per-step gradient metrics for the linearisation arms, encoded as in Figure 3. The arms are drawn on every panel and lie on top of one another. The KD gradient falls twentyfold while the graph gradient stays within a factor of two, so their ratio climbs an order of magnitude and the share of update signs the graph term decides rises from 0.3% to 2%, none of it by design. The cosine oscillates about zero throughout; its excursions coincide across arms because the arms see the same batches.

C SENSITIVITY TO THE GRAPH-LOSS WEIGHT

The weight λ in (2) is the one hyperparameter the method adds. We sweep $\lambda \in \{0.03, 0.1, 0.3\}$ against two references trained through the same stack and evaluated identically: the $\lambda = 0$ control, which is standard knowledge distillation, and supervised fine-tuning on the same prompts. Everything else is held fixed: learning rate 10^{-6} , batch size 32, $\tau = 2$, the same data order, and the unfrozen arg-token and DLA supernodes of Section 3.3.2 with three neurons each (Appendix E has the full table). The graph runs train for 100 steps; the control trains for 300 so that its peak is not cut off. Table 5 and Figure 5 report the result; the references carry three seeds each, the sweep one.

Table 5: Accuracy under each graph-loss weight against the two references, laid out as Table 1: families at step 100, peaks over the run with their step, and the in-distribution dip and recovery. Seed mean $\pm$ sd for the three-seed references; the sweep is one seed each. The control ran to step 300.

	untrained student	SFT (3 seeds)	KD only, $\lambda = 0$ (3 seeds, 300 steps)	$\lambda = 0.03$ (1 seed)	$\lambda = 0.1$ (1 seed)	$\lambda = 0.3$ (1 seed)
22_add (train)	0.653	0.976 ± 0.028	0.900 ± 0.011	0.986	0.983	0.923
222_add	0.190	0.397 ± 0.049	0.370 ± 0.019	0.439	0.470	0.418
2222_add	0.046	0.092 ± 0.015	0.085 ± 0.009	0.107	0.113	0.117
33_add	0.492	0.614 $\pm$ 0.086	0.491 ± 0.010	0.575	0.579	0.521
21_mult	0.135	0.558 ± 0.048	0.530 ± 0.049	0.919	0.938	0.864
mean OOD	0.216	0.415 ± 0.025	0.369 ± 0.022	0.510	0.525	0.480
mean OOD, peak (step)	—	0.440 ± 0.002 (60)	0.383 ± 0.007 (160)	0.516 (85)	0.532 (85)	0.517 (80)
22_add, peak (step)	—	0.993 $\pm$ 0.001 (65)	0.947 ± 0.002 (210)	0.986 (100)	0.986 (70)	0.979 (70)
dip / recovery	—	0.505 / 20	0.248 / 60	0.335 / 35	0.493 / 25	0.609 / 20

Three things hold across every metric and every family. First, every graph run beats both references on every held-out family at step 100, and the margin is not a matter of training length: the control peaks at 0.383 mean OOD at step 160 and declines from there, and SFT at 0.440 at step 60, against 0.532 for $\lambda = 0.1$. Standard distillation is in fact the weaker of the two references here, no better

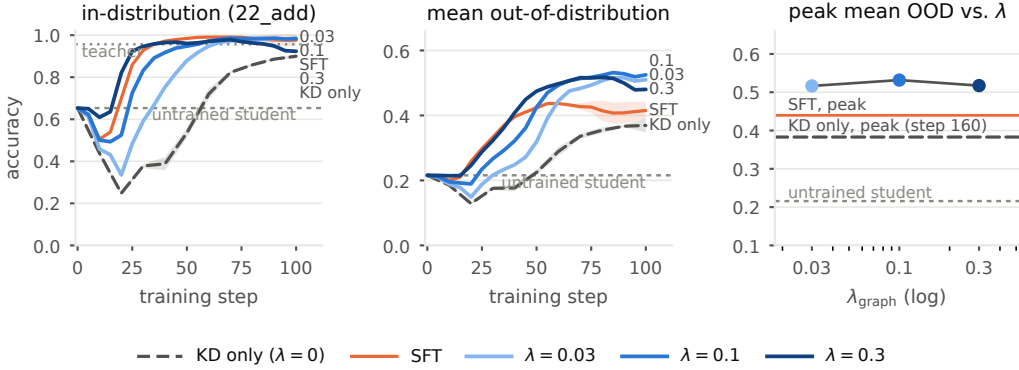


Figure 5: Sensitivity to λ . One hue stepped light to dark with increasing λ ; standard KD in dashed grey and SFT in orange, seed means with a ± 1 sd band, shown to step 100 to share the axis. *Left, centre:* in-distribution and mean out-of-distribution accuracy against training step; the dashed rule is the untrained student and the dotted rule the teacher. *Right:* peak mean out-of-distribution accuracy against λ on a log axis, with the control’s 300-step peak and SFT’s peak as reference rules.

than the untrained student on 33_add at step 100 and never reaching SFT’s peak in 300 steps. Second, every run first falls below the untrained student and then recovers, and the dip is monotone in λ — a minimum of 0.248, 0.335, 0.493 and 0.609 at $\lambda = 0, 0.03, 0.1$ and 0.3 , recovered after 60, 35, 25 and 20 steps — so the dip belongs to the KD objective and the graph term shortens it. SFT, which has no KD term, dips only to 0.505 and recovers by step 20; $\lambda = 0.1$ comes within 0.012 of it. Third, larger λ rises faster but peaks earlier and then declines: $\lambda = 0.3$ peaks at step 70–80 and gives back 0.06 in-distribution and 0.04 OOD by step 100, while $\lambda = 0.03$ and 0.1 are flat from step 85. The response is therefore an inverted U whose top is broad — the three peaks lie within 0.016 of one another across a tenfold change in λ — and what separates the values is stability after the peak rather than the height of it. We use $\lambda = 0.1$ throughout as the largest weight that has not begun to collapse by step 100.

The sweep doubles as a precision check on the training stack. At step one the runs share weights and batch, so any scale-invariant summary of the graph gradient must agree across λ ; both $\|g_{\text{graph}}\|/\lambda$ and $\cos(g_{\text{graph}}, g_{\text{KD}})$ do, to three decimals (2.96 and -0.015). Under the bf16 gradient accumulation of an earlier version of the stack (Appendix A) they did not: the graph gradient, which arrives one prompt at a time into a buffer already holding the far larger KD gradient, was rounded away more severely the smaller λ was, which biased that regime against small λ .

C.1 STRUCTURE CONTROL: A SCRAMBLED TARGET

Does the gain come from the *content* of the teacher’s routing, or would any structured target of the same shape act as a regulariser? We train the $\lambda = 0.1$ configuration against a scrambled target: each row of the teacher’s $K \times K$ supernode adjacency is permuted by a fixed derangement, drawn once from the run seed and reused for every prompt, so the target keeps the teacher’s numbers and per-row sparsity but pairs every supernode with the wrong sources. The run also scores the student against the real target at every step, without gradient. Figure 6 shows the result.

The scrambled target is not merely useless but destructive: in-distribution accuracy falls to 0.036 by step 100 and mean held-out accuracy to 0.084, below the untrained student on every family, while the real target reaches 0.983 and 0.525 from the same initialisation, data order and λ . This is not a matter of gradient scale — the scrambled arm’s graph gradient is *smaller* than the real arm’s at every stage (step-one ratio to the KD gradient 0.0044 against 0.0060, run mean 0.046 against 0.042) — and the student never fits the scrambled target: its loss on it drifts from 0.358 to 0.368 while its loss on the real target stays flat at 0.31–0.33, where the real arm’s falls to 0.276. A routing target the student cannot reach still moves the student, and moves it somewhere the teacher’s target does not. Together with the representation-level baselines of Section 5, which constrain the same residual stream to no effect, this locates the gain in the target’s content.

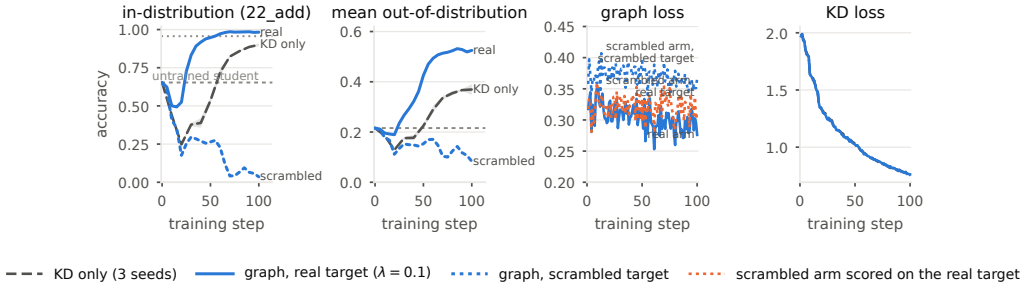


Figure 6: Real against scrambled routing target at $\lambda = 0.1$, single seed, with the KD-only control for reference. *Left pair:* accuracy. *Third:* the graph loss the scrambled arm trains on, the same arm scored against the real target, and the real arm’s loss. *Right:* the KD loss of the three runs.

[pending: seeds 2–3 for $\lambda = 0.1$ and for the scrambled arm; $\lambda = 1$ in the fp32 regime.]

D PERFORMANCE OPTIMISATIONS

An attribution graph costs one batch-expanded forward pass plus a vector–Jacobian product per node, which makes $\mathcal{L}_{\text{graph}}$ far more expensive per prompt than $\mathcal{L}_{\text{KD}}$. Four optimisations bring it inside a training loop. *Probe-set caching:* the residual-stream input to every MLP over all 10^4 prompts is computed once per model and cached to disk, keyed by model and dataset, then read back layer-by-layer, so every activation grid is recovered from the cache rather than by re-running the model. *Analytic pre-selection:* write-vector norms factor as $\|v_i^{\text{out}}\|_2 = |a_i| \|W_{\text{down}}[:, n_i]\|_2$, so the top- k runs in $O(n_{\text{pos}} d_{\text{mlp}})$ space without materialising the $[n_{\text{pos}}, d_{\text{mlp}}, d]$ tensor of write vectors. *Batching:* one batch-expanded forward pass serves a whole batch of target nodes, and all masks and activation grids are scored against each other in a single matmul for (4). *Bounded memory:* only n_{graph} prompts per batch get a graph, and each prompt’s graph loss is backpropagated and freed before the next graph is built, so peak memory is set by one prompt’s attribution graph rather than the batch’s.

[pending: wall-clock and FLOPs per step for each objective, from `--track-flops`.]

E TRAINING AND EVALUATION DETAILS

Table 6: Configuration shared by every run in the paper unless a section says otherwise. The three trainers (SFT, standard KD, graph distillation) share one implementation of the optimiser, precision regime, data order and evaluation.

teacher / student	Meta-Llama-3-8B-Instruct / Llama-3.2-1B-Instruct, 1.24B parameters
training data	22_add prompts “ $a+b=$ ”, batch 32, 100 steps (control: 300)
optimiser	AdamW, learning rate 10^{-6} , no schedule, gradient clipping at 1.0
precision	fp32 master weights and optimiser moments, bf16 autocast forward; teacher bf16
KD term	$\tau = 2$, forward KL over the positions predicting the answer tokens and EOS
graph term	$\lambda = 0.1$, JSD, arg-token + DLA supernodes, 3 neurons per label
attribution	top 10% of neurons per layer by write norm in the teacher, 1% in the student; logit targets: top tokens covering 0.95 of $\text{softmax}(z/\tau)$, at most ten
linearisation	unfrozen (full Jacobian) unless stated; Appendix B
evaluation	greedy decoding, up to 5 new tokens, first integer of the continuation; every 5 steps (control: 10) on 22_add and the four held-out families
test sets	5000 prompts per family (1000 for 21_mult), disjoint from training
baselines	TinyBERT recipe (MSE on projected hidden states and on attention patterns, uniform layer map) and linear CKA on residual streams; weight 0.1 on the KD term
seeds	1 for graph runs; 1, 2, 3 for SFT, the KD control and the two representation baselines

Evaluation. An answer is scored by decoding only the generated continuation and parsing its leading integer. This matters more than it sounds: Llama 3 tokenises digits in groups of one to three, and the untrained student often emits an answer digit by digit, so a one-token budget truncates it, while parsing the prompt echo lets a second, echoed equation override a correct answer. Both effects moved every family’s accuracy, the untrained student’s included, so all numbers in the paper come from one evaluation convention and are not comparable to earlier drafts.

Held-out families. 222_add and 2222_add are three- and four-operand sums of two-digit numbers, 33_add is three-digit addition (the only family whose answers need two tokens), and 21_mult is a two-digit by one-digit product.

Pending results. The following slots are marked *[pending]* in the text and will be filled from the same scripts without editing the tables by hand: seeds 2–3 for the $\lambda = 0.1$ run and for the scrambled arm (Section 5, Appendix C); the attention-only and RMSNorm-only linearisation arms and seeds for the both-frozen arm (Appendix B); the fp32 re-measurement of Table 2; FLOPs per step (Appendix D).

[Still to write: the complete ANOVA category list including the composite and combination-sum categories omitted from Section 3.3; additional supergraph figures; per-supernode activation-heatmap montages.]